

Einzelreflexion – Projektarbeit

Eigener Beitrag - Beispiele:

Aufgabe:	Beispiel / Beleg: Commit-Historie
1. Erstellung der Auth-Services für die Benutzerverwaltung	Beispiel-Commit
2. Implementierung der API-Endpunkte für Gewohnheiten und Useraktionen	API-Endpunkte: Beispiel-Code
3. Implementierung der Swagger-UI zur Dokumentation der API	Pull-Request 163 http://localhost:3001/api/docs/
4. Durchführung von Tests und Bugfixing	Bugfix-Beispiel
5. Code-Reviews bei sämtlichen Pull-Requests	Liste Pull-Requests
6. Erstellung von Dev-Dokumentation und Bash-Skripten	test-backend.sh dev-db-setup.sh src/docu (interne Entwicklerdoku)

Welche Aufgaben lagen, klar in meiner Verantwortung?

Abgesehen von den allgemeinen Teamaufgaben, die überwiegend in Gruppen von zwei bis drei Personen bearbeitet wurden, lag die Verantwortung für die Authentifizierungs- und Autorisierungsdienste sowie die API-Dokumentation klar bei mir. Darüber hinaus habe ich eigenständig die Implementierung und das Testen der Endpunkte für Gewohnheiten und Useraktionen übernommen. Zusätzlich trug ich die Verantwortung für die von mir übernommenen Dev-Tasks und setzte diese eigenständig um.

Wo habe ich andere unterstützt, ohne die Hauptverantwortung zu tragen?

Ich habe an sämtlichen Code-Reviews teilgenommen und andere Teammitglieder im Rahmen von Pair-Programming unterstützt. Gemeinsam mit Tristan habe ich am Backend-Refactor gearbeitet, bei dem wir von einer einfachen relationalen SQL-Datenbank zu einer Lösung mit Prisma ORM gewechselt sind, um komplexere Zusammenhänge besser abbilden zu können. Zusätzlich unterstützte ich das Team durch Recherchen und technische Vorschläge zur Umsetzung der Anforderungen aus den User Stories. Auf dieser Basis entschieden wir uns unter anderem für den Einsatz von Node.js, Docker, Supabase und Swagger UI. Des Weiteren habe ich in Zusammenarbeit mit dem Team die Dokumentation im Ordner /docs, sowie team-interne Dokumentation im Ordner /src/docu erstellt und gepflegt.

Bezug zu Aufgaben anderer (vor- / nachgelagert)

Meine Arbeit im Backend stellte die Grundlage für die Frontend-Entwicklung dar. Insbesondere die Authentifizierungsmechanismen sowie die implementierten API-Endpunkte wurden direkt von den Frontend-Komponenten genutzt. Der Backend-Refactor mit Prisma ORM ermöglichte es zudem, weitere Funktionen stabil und konsistent umzusetzen.

Problemlösung & Reflexion

Was hat gut funktioniert? Warum?

Als Team konnten wir auch komplexe technische Aufgaben durch gute Kommunikation und regelmäßige Programming-Sessions erfolgreich lösen. Das Entwirren einer anfangs chaotischen Codebasis mit kollidierenden Arbeitsständen führte durch die Zusammenarbeit mit Tristan und Patrick zu einer klar strukturierten Architektur, die sich an im Modul Software Engineering vermittelten Best Practices orientiert.

Die Entwicklung des Backends als Unterstützung für die Frontend-Entwicklung funktionierte durch die enge Abstimmung mit Patrick sehr gut. Auch die Implementierung der notwendigen Funktionen, um die in den User Stories beschriebenen Anforderungen über den gesamten Tech-Stack hinweg abzubilden, verlief dadurch effizient. Die Entwickler im Frontend erstellten Dev-Tasks für die benötigten Backend-Funktionalitäten, welche dann von uns umgesetzt wurden. Die Teamarbeit mit Tristan zur Implementierung von Prisma ORM im Backend-Refactor hat sehr gut funktioniert, da wir uns gut ergänzten und gemeinsam Lösungen für auftretende Probleme fanden.

Name: Paul Händler
Matrikelnummer: s87141
Studiengang / Modul: Wirtschaftsinformatik / Software Engineering 2
Projektname: Gewohnheitstier (Gruppe 3B)

Was hat nicht gut funktioniert? Warum?

Die anfängliche Implementierung auf Basis einer rein relationalen Datenbank erwies sich als suboptimal, da komplexe Beziehungen zwischen Nutzern und Gewohnheiten nur schwer abbildbar waren. Ein Feature, das letztlich aus zeitlichen Gründen nicht implementiert werden konnte, war die Möglichkeit, für einzelne Gewohnheiten unterschiedliche Wiederholungsraten und Bedingungen zu definieren. Dieses Feature wäre mit einer NoSQL-Datenbank wie MongoDB oder durch eine frühere Nutzung von Prisma ORM deutlich einfacher umzusetzen gewesen. Zusätzlich war es organisatorisch schwierig, im Team eine klare Arbeitsverteilung und einheitliche Dokumentation zu etablieren. Es wurden viele unterschiedliche Tools verwendet (Miro, Discord, WhatsApp, GitHub, Meeting-Protokolle, Timeful), aber nicht alle wurden konsequent genutzt, was die Übersicht und Abstimmung manchmal erschwerte.

Rückblick: Was würde ich anders machen?

Wenn ich das Projekt erneut starten würde, hätte ich mich von Anfang an für eine NoSQL-Datenbank wie MongoDB oder direkt für Prisma ORM entschieden. So hätten wir uns die spätere Umstellung der PostgreSQL-Datenbank auf Prisma ORM sparen können und mehr Zeit für die Implementierung neuer Funktionen gehabt.

Außerdem hätte ich von Beginn an eine klarere Trennung zwischen Backend und Frontend angestrebt. Die Umsetzung der Separation of Concerns und einer sauberen Architektur hätte ich gern zu Beginn ordnungsgemäß umgesetzt, statt diese nachträglich aufwendig zu refactoren. Zu diesem Zeitpunkt fehlte mir jedoch noch wichtiges Fachwissen. Auch hier hätten wir Zeit sparen können, die wir in weitere Funktionalitäten investiert hätten. Zudem hätte ich die Definition und konsequente Einhaltung gemeinsamer Konventionen im Team früher eingeführt und mich selbst stärker an diese gehalten. Mit der Einführung des Pair-Programming und der Code-Reviews wurden wir zu einem eingespielten Team, das effizient zusammenarbeiten konnte. Hierbei habe ich viel gelernt, und durch den Input meiner Teamkollegen konnte ich Probleme oft schneller lösen. Besonders Tristan war hier eine große Hilfe. Unser Fortschritt hätte deutlich zügiger sein können, wenn unsere Programming-Sessions von Anfang an stattgefunden hätten.

Einordnung in den Software-Engineering-Prozess

Meine Tätigkeiten lassen sich folgenden Phasen zuordnen:

Analyse: Analyse der User Stories und Evaluation geeigneter Technologien (z. B. Supabase, Docker)

Entwurf / Design: Entwurf des API-Designs, der Authentifizierungskonzepte & ORM Schemata

Implementierung: Implementierung der Auth-Services, API-Endpunkte, Swagger-UI

Test / Qualitätssicherung: Erstellung einfacher Unit-Tests, Durchführung von Integrationstests & Code-Reviews

Persönliches Fazit

Fachlicher Lernzuwachs

Ich habe meine Kenntnisse in der Backend-Entwicklung mit Node.js und Express deutlich vertieft. Insbesondere habe ich viel über Benutzer-Authentifizierung und Autorisierung gelernt. Zusätzlich habe ich praktische Erfahrungen mit Supabase als Backend-as-a-Service gesammelt, was mir geholfen hat, die Integration von Datenbanken in Webanwendungen besser zu verstehen. Die Arbeit mit Pull-Requests, Dev-Tasks und Tests/Swagger-UI hat mir verdeutlicht, wie wichtig eine gute Dokumentation für Teamarbeit und Wartbarkeit ist. Darüber hinaus hat das aufwändige Refactoring, um die angestrebte Trennung von Backend und Frontend zu erreichen, mein Verständnis für Softwarearchitekturen im Web deutlich verbessert.

Methodischer / organisatorischer Lernzuwachs

Ich habe viel über Teamarbeit und den Umgang mit Git-Versionskontrolle gelernt. Zu Beginn war es schwierig, den Überblick über verschiedene Entwicklungsstände zu behalten, da ohne klare Regeln gearbeitet wurde. Mit der Zeit einigten wir uns auf eine klare Branch-Strategie und feste Regeln für das Mergen in den Hauptbranch, was die Zusammenarbeit deutlich verbesserte.

Beitrag zum Projekterfolg

Mein Beitrag zum Projekt war umfangreich, hätte jedoch zu Beginn stärker an Best Practices ausgerichtet sein können. Nachdem wir uns etwa im ersten Drittel des Semesters besser koordiniert hatten, wurden wir deutlich produktiver.

Mein Schwerpunkt lag gegen Ende vor allem in der Backend-Entwicklung, dennoch habe ich an allen Teilen des Projekts mitgewirkt und das Team bestmöglich unterstützt. Meine Arbeit hat dazu beigetragen, eine funktionierende Webanwendung zu entwickeln, die den Anforderungen des Moduls Software Engineering 2 gerecht werden sollte.